COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	10/14/2026	**DATE SUBMITTED:**	
NAME:	Julian Benedict D.R. Barizo Justin Jake Macatangay	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 12.1 Algorithmic Strategies

1. PROCEDURE OUTPUT

ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)

The Problem: MINIMUM STEPS TO ONE

On a positive integer, you can perform any one of the following 3 steps.

1.) Subtract 1 from it. (n = n - 1)
2.) If its divisible by 2, divide by 2. (If n % 2 == 0, then n = n / 2)
3.) If its divisible by 3, divide by 3. (If n % 3 == 0, then n = n / 3).

Now the question is, given a positive integer n, find the minimum number of steps that takes n to 1.

Examples:
1.) For n = 1, output: 0
2.) For n = 4, output: 2 (4 /2 = 2 /2 = 1)
3.) For n = 7, output: 3 (7 -1 = 6 /3 = 2 /2 = 1)

One can think of greedily choosing the step, which makes n as low as possible and conitnue the same, till it reaches 1. If you observe carefully, the greedy strategy doesn't work here.

Table 12-1 Algorithmic Strategies and Examples

Strategy	Algorithm	Analysis
Recursion	Recursive Array Binary Search	This is recursive since it splits the array in half and calls itself inside its own body to search for the elements on the left and right side of the array. To further explain, if the element that it's finding is on the left half, it calls itself with an updated lower bound. While if it's

		on the right side it calls itself with a higher bound. Lastly, the recursive function contains a base case where if the middle element matches the target or if the search becomes invalid (low > high), the code breaks or prints an output depending on the situation, and exits if the base case's requirements are met.
Brute Force	Linear Search / Bubble Sort	Evaluate every possible element or combination sequentially without optimization until the matching target or correct order is found.
Backtracking	Inorder Traversal	Inorder traversal contains backtracking due to the fact that it goes down to the bottom of the tree and if it sees that it cannot go any deeper, it moves on to the next line and prints that node's key. After printing, the algorithm checks for a right sibling (nextChild). If there are no more children to explore, the function finishes and returns to the parent node. This return action pops the current call off the system call stack, backtracking up to the previous level to resume execution.
Greedy	Selection Sort	This qualifies as a greedy algorithm because it tries to solve a problem by making the locally optimal choice at each stage of the traversal in the supposed array with the hope of finding the global optimal solution.
Divide-And-Conquer	Merge Sort / Quick Sort	Divides the main problem into smaller independent subproblems, solve them recursively and combine their results to solve the original problem.

Table 12-2. Memoization Implementation

Memoization

```
int memo[n+1]; // we will initialize the elements to -1 ( -1 means, not solved it yet )
int getMinSteps ( int n )
{
    if ( n == 1 ) return 0; // base case
    if( memo[n] != -1 ) return memo[n]; // we have solved it already :)
    int r = 1 + getMinSteps( n - 1 ); // '-1' step . 'r' will contain the optimal answer finally
    if( n%2 == 0 ) r = min( r , 1 + getMinSteps( n / 2 ) ) ; // '/2' step
    if( n%3 == 0 ) r = min( r , 1 + getMinSteps( n / 3 ) ) ; // '/3' step
    memo[n] = r ; // save the result. If you forget this step, then its same as plain recursion.
    return r;
}
```

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

int getMinSteps (int n, std::vector<int>& memo) {

    if ( n == 1 ) {
        return 0; // base case
    }
    if( memo[n] != -1 ) {
        return memo[n];
        }  // we have solved it already :)
    int r = 1 + getMinSteps( n - 1, memo );  // '-1' step . 'r' will contain the optimal answer finally

    if( n%2 == 0 ) {
        r = std::min( r , 1 + getMinSteps( n / 2, memo ) ) ; // '/2' step
    }
    if( n%3 == 0 ) {
        r = std::min( r , 1 + getMinSteps( n / 3, memo ) ) ; // '/3' step
    }

    memo[n] = r ; // save the result. If you forget this step, then its same as plain recursion.
    return r;
};
```

```cpp
//int main function

int main() {
    int n;

    std::cout<<"Enter a positive integer: ";
    std::cin>>n;

    if (n < 1) {
        std::cout<<"PLease enter a positive integer!";
        return 0;
    };

    std::vector<int> memo(n+1, -1);

    std::cout<<"The minimum steps are: " << getMinSteps(n, memo) << std::endl;

    return 0;

};
```

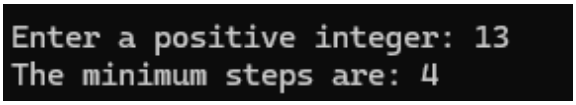
Screenshot	
Analysis	This algorithm checks for the minimum possible steps for an integer to go to 1. It starts by recursively subtracting one, dividing by two, or dividing by three while storing the optimal result of each integer state in a lookup array to eliminate redundant subproblems. By evaluating all valid transitions and using cached answers, the program guarantees a globally optimal solution in **O(n)** time, as seen when reducing 13 down to 1 in four steps.

Table 12-3. Bottom-Up Dynamic Programming Implementation

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int getMinSteps(int n) {
    vector<int> dp(n + 1, 0);

    dp[1] = 0;

    for (int i = 2; i <= n; i++) {
        dp[i] = 1 + dp[i - 1];

        if (i % 2 == 0) {
            dp[i] = min(dp[i], 1 + dp[i / 2]);
        }

        if (i % 3 == 0) {
            dp[i] = min(dp[i], 1 + dp[i / 3]);
        }
    }

    return dp[n];
}
```

```cpp
int main() {
    int testCases[] = {1, 4, 7, 10};

    cout << "=== Table 12-3. Bottom-Up Dynamic Programming Implementation ===" << endl;
    for (int n : testCases) {
        cout << "n = " << n << " -> Minimum Steps: " << getMinSteps(n) << endl;
    }

    return 0;
}
```

Screenshot	`=== Table 12-3. Bottom-Up Dynamic Programming Implementation ===`
	`n = 1 -> Minimum Steps: 0`
	`=== Table 12-3. Bottom-Up Dynamic Programming Implementation ===`
	`n = 1 -> Minimum Steps: 0`
	`n = 4 -> Minimum Steps: 2`
	`n = 1 -> Minimum Steps: 0`
	`n = 4 -> Minimum Steps: 2`
	`n = 4 -> Minimum Steps: 2`
	`n = 7 -> Minimum Steps: 3`
	`n = 10 -> Minimum Steps: 3`

Analysis	The bottom-up dynamic programming approach solves the problem iteratively by building an array from $i = 2$ up to $i = n$ using previously computed optimal subproblems. This achieves $O(n)$ time and space complexity while avoiding recursive call stack overhead and stack overflow risks.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C: Create C++ code to implement various algorithmic strategies, Create C++ code to solve problems using learned algorithmic strategies

Pseudocode

```
FUNCTION countPaths(mat, i, j, cost)
   // Out of bounds, or remaining cost already negative
   IF i < 0 OR j < 0 OR cost < 0 THEN
      RETURN 0
   END IF

   // Reached the starting cell
   IF i = 0 AND j = 0 THEN
      IF mat[0][0] = cost THEN
         RETURN 1
      ELSE
         RETURN 0
      END IF
   END IF

   // Paths into (i, j) come from the cell above or the cell to the left
   remaining ← cost - mat[i][j]
   fromAbove ← countPaths(mat, i - 1, j, remaining)
   fromLeft  ← countPaths(mat, i, j - 1, remaining)

   RETURN fromAbove + fromLeft
END FUNCTION


PROGRAM main
   INPUT M, N
   INPUT matrix mat of size M × N
   INPUT cost

   result ← countPaths(mat, M - 1, N - 1, cost)
   OUTPUT "Number of paths with cost", cost, ":", result
END PROGRAM
```

Solve by hand

Cost	Paths	Count
21	RRDDRD (4+7+1+3+1+2+3)	1

26	RRDDDR (4+7+1+3+1+7+3), DRRDRD (4+6+7+3+1+2+3)	2
27	RDRDRD, DDRRRD	2
29	RRDRDD	1
31	DRRDDR, DRDRRD, DDDRRR	3
32	RRRDDD, RDRDDR, RDDRRD, DDRRDR, DDRDRR	5
34	DRRRDD	1
35	RDRRDD	1
36	DRDRDR, DRDDRR	2
36	RDDRDR, RDDDRR	2

Code

HOA12 > ILO_C_Macatangay_Barizo.cpp > main()

```cpp
#include <iostream>
#include <vector>
using namespace std;


// Counts paths from (0,0) to (i,j) whose total cost equals 'cost'.
// Works backwards from the destination, subtracting each cell's value.
int countPaths(const vector<vector<int>>& mat, int i, int j, int cost) {
    if (i < 0 || j < 0 || cost < 0)
        return 0;


    if (i == 0 && j == 0)
        return (mat[0][0] == cost) ? 1 : 0;


    return countPaths(mat, i - 1, j, cost - mat[i][j]) +
           countPaths(mat, i, j - 1, cost - mat[i][j]);
}


int main() {
    int M, N, cost;
    cout << "Enter rows and columns: ";
    cin >> M >> N;


    vector<vector<int>> mat(M, vector<int>(N));
    cout << "Enter the matrix:\n";
    for (int i = 0; i < M; i++)
        for (int j = 0; j < N; j++)
            cin >> mat[i][j];


    cout << "Enter target cost: ";
    cin >> cost;


    cout << "Number of paths with cost " << cost << ": "
         << countPaths(mat, M - 1, N - 1, cost) << endl;
    return 0;
}
```

Output

```
Enter rows and columns: 4 4
Enter the matrix:
4 7 1 6
6 7 3 9
3 8 1 2
7 1 7 3
Enter target cost: 26
Number of paths with cost 26: 2
```

Analysis	
Barizo	**This code counts the number of possible paths from the first cell to the last cell of a matrix with a given total cost. It uses recursion to check paths by moving upward or leftward while subtracting each cell's value from the target cost. Finally, it displays the total number of valid paths that match the given cost.**
Macatangay	**The recursive algorithm calculates path counts by exploring rightward and downward moves while deducting cell costs from the target sum until reaching the final cell. Base conditions stop execution if the cost becomes negative, moves out of matrix bounds, or reaches the destination with a matching sum. It runs with $O(2^{M+N})$ time complexity due to exponential branching and uses $O(M + N)$ space for the recursive stack.**

Conclusion

Macatangay

This activity showed basic algorithmic techniques like recursion, brute force, backtracking, divide and conquer, and dynamic programming; and showed that greedy algorithms are not always successful – for example, in the problem Minimum Steps to One. Comparing Memoization and Bottom Up DP brought into light that iterative table filling has the benefit of lack of call stack overhead and a prevention from stack overflow errors. In addition, recursive matrix problem demonstrated how target cost pathfinding can be decomposed into subproblems: right and down, using recursive decision trees. In general i was able to run all the C++ programs and understand the approaches, but can do better using memoization for recursive grid problems to make it more efficient.

Barizo

In this activity, I learnt about different algorithmic strategies, such as recursion, brute force, backtracking, greedy algorithms, and dynamic programming. I also learned how to apply memoization and bottom-up approaches to solve problems efficiently. Overall, this activity improved my understanding of problem-solving using C++.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Generation of Ideas, and Code Guidance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.